

Self Organization in Networking(self organizing system missing in this topic)

Introduction to Self Organization:

1. What's the idea of self-organized (SO) systems?

The idea of self-organized (SO) systems revolves around the concept of systems that spontaneously organize themselves without centralized control or external intervention. In SO systems, individual components interact locally based on simple rules or principles, leading to the emergence of complex and adaptive behaviours at the system level.

2. Why is SO so popular for modern systems?

Self-organized (SO) systems are popular for modern systems due to several reasons:

- **Flexibility:** SO systems can adapt to changing conditions and environments without the need for explicit programming or centralized coordination.
- **Robustness:** The distributed nature of SO systems often leads to increased resilience and fault tolerance, as they can continue functioning even if individual components fail or are removed.
- **Scalability:** SO systems can scale effectively to accommodate changes in size or complexity, making them suitable for large-scale distributed environments.
- **Efficiency:** By leveraging local interactions and decentralized decision-making, SO systems can achieve efficient resource allocation and optimization.

3. What are the properties of SO systems?

Decentralization: SO systems lack centralized control, with decision-making distributed among individual components.

Emergence: Complex global behaviors and patterns arise from the interactions of simple local rules followed by individual components.

Adaptation: SO systems can adjust their structure and behavior in response to changes in their environment or internal conditions.

Robustness: SO systems exhibit resilience to disturbances or perturbations, often through redundancy or self-repair mechanisms.

Autonomy: Individual components in SO systems operate autonomously, with limited or no coordination from external entities.

4. What are the limits of SO systems?

Complexity management: As SO systems scale or become more complex, managing interactions and dependencies among individual components can become challenging.

Lack of global optimization: While SO systems can exhibit emergent properties, they may not always achieve globally optimal solutions, leading to suboptimal outcomes in certain scenarios.

Vulnerability to perturbations: Although SO systems are often robust, they may still be vulnerable to certain types of disturbances or failures, especially if critical dependencies exist among components.

Coordination overhead: In some cases, achieving effective coordination among autonomous components in SO systems may require additional communication or synchronization mechanisms, introducing overhead and complexity.

5. What does emergence mean?

Emergence refers to the phenomenon where complex global patterns, behaviors, or structures arise from the interactions of simple local entities or rules in a system. In self-organized (SO) systems, emergence is a fundamental property, leading to the spontaneous creation of organized structures or behaviours without centralized control or explicit programming. These emergent properties often cannot be predicted or directly attributed to any individual component but result from the collective behavior of the system as a whole.

Design of Self Organizes Systems:

1. Identify important design principles of self-organized systems.

Decentralization: Distributing decision-making and control among individual components rather than relying on centralized authority.

Local interactions: Components interact based on local information and simple rules, leading to emergent global behaviours.

Adaptation: Systems adjust their behavior in response to changes in their environment or internal conditions.

Robustness: Systems exhibit resilience to disturbances or failures, often through redundancy or self-repair mechanisms.

Emergence: Complex global patterns or behaviours arise from the interactions of simple local entities, without explicit programming or control.

2. Identify the design paradigms for algorithms of self-organized systems.

Design paradigms for algorithms of self-organized systems include:

Swarm intelligence: Inspired by collective behaviours observed in natural systems like ant colonies or bird flocks, swarm intelligence algorithms coordinate the actions of individual agents to achieve collective goals.

Evolutionary computation: Borrowing principles from biological evolution, evolutionary algorithms use mechanisms such as selection, mutation, and reproduction to iteratively improve solutions to optimization problems.

Cellular automata: Modeling systems as grids of cells that evolve over discrete time steps according to local rules, cellular automata algorithms exhibit emergent behaviors and self-organization.

Agent-based modeling: Simulating systems as collections of autonomous agents that interact with each other and their environment, agent-based models capture complex dynamics and emergent phenomena.

3. How are these paradigms applied to communication protocols?

These paradigms are applied to communication protocols in various ways:

Adaptive routing: Self-organized routing protocols dynamically adjust network paths based on local observations and interactions among network nodes, optimizing traffic flow and resource utilization.

Medium access control (MAC): Self-organized MAC protocols coordinate access to shared communication channels among multiple users or devices, adapting contention and scheduling strategies based on local channel conditions.

Network formation: Self-organized protocols enable autonomous devices to form ad hoc networks or organize into hierarchical structures without centralized coordination, facilitating communication in dynamic or decentralized environments.

Fault tolerance: Self-organized communication protocols incorporate mechanisms for fault detection, isolation, and recovery, allowing networks to maintain connectivity and functionality even in the presence of failures or disruptions.

4. Design Paradigms for Self-organized Systems.

5. Paradigm #1: Design local behavior rules that achieve global properties.

6. Paradigm #2: Do not aim for perfect coordination: exploit implicit coordination.

7. **Paradigm #3: Minimize long-lived state information.**

8. **Paradigm #4: Design protocols that adapt to changes.**

Applications of Self-organization to Networking:

1. **Why are the different examples considered examples of SO? Which aspect exactly is considered self-organizing.**

The different examples are considered examples of self-organizing (SO) systems because they exhibit properties of self-organization, where complex global behaviors or structures emerge from the interactions of simple local entities or rules without centralized control. The aspect considered self-organizing in each example varies but generally includes:

Adaptive Routing
Medium Access Control
Network Formation
Fault Tolerance

The different examples are considered examples of self-organizing (SO) systems because they exhibit decentralized decision-making and emergent behaviors stemming from interactions among individual components. The self-organizing aspect lies in how the systems autonomously adapt, coordinate, or organize themselves without centralized control or explicit instructions.

2. **Identify how feedback is implemented in the different applications of SO; identify positive and negative feedback separately.**

Adaptive Routing

Positive feedback: Positive feedback mechanisms in adaptive routing may involve amplifying the use of routes that have recently demonstrated high performance, such as low latency or high throughput. This encourages the continued selection of efficient paths, potentially leading to congestion if not balanced with negative feedback mechanisms.

Negative feedback: Negative feedback in adaptive routing aims to regulate traffic flow and prevent congestion by dynamically adjusting route selections based on congestion indicators or packet loss rates. For example, routers may avoid routes experiencing high congestion, redirecting traffic to less congested paths.

Medium Access Control

Positive feedback: Positive feedback in MAC protocols may involve increasing contention window sizes for devices that successfully transmit data packets, allowing them to access the medium more frequently. This can lead to increased channel utilization but may also contribute to unfairness if not balanced with negative feedback mechanisms.

Negative feedback: Negative feedback in MAC protocols helps regulate channel access and mitigate collisions by reducing contention window sizes or backoff intervals for devices experiencing collisions or contention. This helps prevent channel saturation and ensures fair access to the medium among competing devices.

Network Formation

Positive feedback: Positive feedback mechanisms in network formation may involve reinforcing connections between devices that successfully exchange data packets or maintain stable communication links. This can lead to the formation of dense clusters or hubs within the network, enhancing connectivity but potentially increasing redundancy

Negative feedback: Negative feedback in network formation aims to balance network connectivity and resource utilization by limiting connections or pruning redundant links. For example, devices may avoid forming connections with neighbouring devices that offer little additional benefit or contribute to network congestion

Fault Tolerance

Positive feedback: Positive feedback in fault tolerance mechanisms may involve reinforcing successful recovery actions or alternative communication paths that effectively restore network connectivity. This encourages the continued use of reliable recovery strategies and strengthens the network's resilience to failures.

Negative feedback: Negative feedback in fault tolerance mechanisms helps prevent excessive resource consumption or overreaction to transient faults by limiting the scope or frequency of recovery actions. For example, devices may delay recovery attempts to allow temporary disturbances to resolve on their own, avoiding unnecessary disruptions

3. Identify the implementation of communication in the example applications.

Adaptive Routing

Implementation: In adaptive routing, communication occurs between network nodes to exchange routing information, such as link state updates, routing table entries, or path advertisements. This communication enables nodes to share information about network topology changes, link conditions, and available routes.

Protocols: Common routing protocols such as OSPF (Open Shortest Path First) or BGP (Border Gateway Protocol) are used to facilitate communication and exchange routing information between routers. These protocols employ various mechanisms, including flooding, link-state updates, or path advertisements, to disseminate routing information and compute optimal paths through the network.

Medium Access Control

Implementation: In MAC protocols, communication occurs between devices competing for access to a shared communication medium, such as a wireless channel or a shared bus. Devices contend for channel access based on predefined rules or contention mechanisms, such as carrier sense multiple access (CSMA) or contention-based access methods

Protocols: MAC protocols define rules and procedures for channel access, collision avoidance, and contention resolution among competing devices. Examples of MAC protocols include CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) used in wireless networks like Wi-Fi, and CSMA/CD (Carrier Sense Multiple Access with Collision Detection) used in wired Ethernet networks.

Network Formation

Implementation: In network formation, communication enables devices to discover neighbouring nodes, exchange network topology information, and establish connections or links with other devices. This communication allows devices to self-organize into network structures, such as ad hoc networks, sensor networks, or mesh networks.

Protocols: Various protocols and mechanisms are used for neighbour discovery, link establishment, and network formation in self-organizing networks. Examples include protocols like AODV (Ad hoc On-Demand Distance Vector) or OLSR (Optimized Link State Routing) used in mobile ad hoc networks (MANETs), and Zigbee or Bluetooth protocols used in wireless sensor networks or personal area networks (PANs)

Fault Tolerance

Implementation: In fault-tolerant systems, communication enables nodes to detect faults, exchange fault-related information, and coordinate recovery actions. This communication allows the system to detect and respond to failures, reroute traffic, or activate redundant resources to maintain operational integrity

Protocols: Fault-tolerant systems employ various communication protocols and mechanisms for fault detection, isolation, and recovery. These may include protocols for heartbeat monitoring, failure detection, message logging, or distributed consensus algorithms such as Paxos or Raft. Additionally, protocols like Virtual Router Redundancy Protocol (VRRP) or Hot Standby Router Protocol (HSRP) are used for fault tolerance in routing infrastructure.

4. Identify how stochastic aspects are integrated each of the examples.

Adaptive routing: Through stochastic routing decisions, randomized route exploration, and probabilistic load balancing to optimize traffic distribution and adapt to network conditions.

Medium access control (MAC): Using stochastic backoff mechanisms, randomized contention windows, and probabilistic access schemes to optimize channel utilization and fairness in channel access.

Network formation: Employing stochastic neighbor discovery, randomized link establishment, and probabilistic clustering to adaptively organize devices and form network structures efficiently.

Fault tolerance: Incorporating stochastic fault detection, randomized fault recovery, and probabilistic redundancy management to enhance system resilience and adaptability to fault conditions.

5. Identify which level of the design paradigm 4 is implemented by the different examples.